Relatório de Arquitetura e IA

Squad 15 - Gráfica Máxima

1. UX/UI para Interfaces Inteligentes

1.1. Wireframes e Fluxos de Conversação

A interface adota um layout **Dashboard Operacional** com navegação lateral fixa (Sidebar) recolhível e cabeçalho (Topbar) contextual. A hierarquia prioriza dados operacionais e tomadas de decisão rápidas:

- **Topbar:** Indicador de status dos serviços (Backend e APIs de IA), notificações do sistema e busca global.
- **Sidebar:** Acesso aos módulos centrais (Dashboard, Orçamentos, Cadastro, Papel, Configurações).
- **Área de Trabalho:** Dividida entre KPIs operacionais, painel dinâmico de ação e um Drawer contextual com o Assistente de Inteligência Artificial.

Componentes de Inteligência Artificial (IA):

- **Copilot Drawer:** Janela retrátil de assistência contextual para alternar entre preenchimento manual e sugestões da IA.
- **Smart Prompt Card:** Campo com suporte a upload de arquivos (drag-and-drop de briefings) para extração via OCR e LLM.
- **Approval Trigger (Human-in-the-Loop):** Botão de "Revisar e Aplicar Sugestão", garantindo que a IA não altere o faturamento sem chancela humana.

1.2. Design System e Acessibilidade

O Design System segue a identidade visual da Gráfica Máxima, utilizando variáveis de contraste seguro. A acessibilidade (WCAG 2.1) foi priorizada, garantindo navegação total por teclado (tabindex estruturado) e suporte a leitores de tela com regiões `aria-live` para o streaming de respostas da IA.

2. Arquitetura de Software e Stack de Desenvolvimento

2.1. Escolha da Stack e Estratégia de IA

O projeto utiliza um frontend em HTML5/JS moderno modularizado e um backend Node.js atuando como API Proxy. As IAs foram distribuídas da seguinte forma, conforme planejamento estratégico do Squad 15:

- **ChatGPT (OpenAI) - Paga:** Criação de documentação, estruturação de código, modelagem das regras de negócio e geração da base arquitetural.
- **Gemini (Google) - Paga:** Refatorações avançadas de código, análise de integrações de rede e auditoria de UX/UI.
- **Claude (Anthropic) - Gratuita:** Acompanhamento geral, correções sintáticas rápidas e sugestões pontuais de otimização no frontend.

3. Tratamento de Latência, Erros e Indisponibilidade

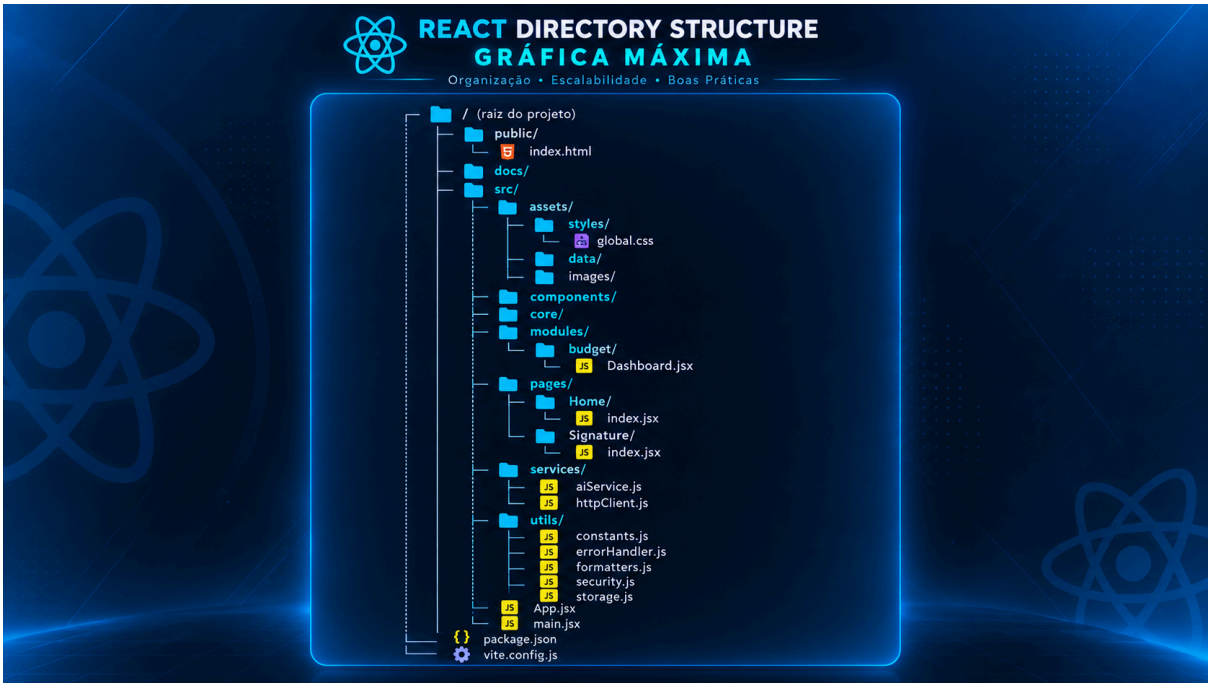
Como o sistema lida com orçamentos em tempo real, a resiliência é crítica. Implementamos o padrão **Circuit Breaker** e **Retry com Backoff** no arquivo `errorHandler.js`.

- **Latência (Timeout):** Requisições para IA possuem um teto de 10 segundos. Caso excedam, o `AbortController` cancela a requisição e exibe um aviso amigável.
- **Indisponibilidade (Circuit Breaker):** Se a API de IA falhar 3 vezes consecutivas, o circuito "abre", e o sistema passa a utilizar a Tabela de Custos Estática armazenada localmente (Fallback Gracioso).

4. Estrutura do Código

4.1. Visão Geral de Diretórios

A arquitetura modular isola os serviços externos e protege as chaves das APIs de inteligência artificial no Backend Proxy. A estrutura segue o padrão **Feature-Driven**:/site



4.2. Adicionar uma Imagem no Sistema

No padrão estrutural definido, as imagens (logotipos, ícones de papel, marca d'água) devem residir estritamente no diretório `/assets/img/`. Para adicionar dinamicamente ou estaticamente uma imagem na interface, o fluxo deve seguir as diretrizes do Design System

[ÁREA RESERVADA PARA O DIAGRAMA DO DIRETÓRIO /assets/img]
Insira aqui o diagrama visual da estrutura de imagens gerado pela equipe de design.

Exemplo de Implementação de Imagem via JavaScript Modular:

JavaScript

```
// Arquivo: js/modules/dashboard.js
export function renderizarLogoEmpresa(containerId) {
    const container = document.getElementById(containerId);
    if (container) {
        const logoImg = document.createElement("img");
        logoImg.src =
"../../assets/img/logo-grafica-maxima.png";
```

```
        logoImg.alt = "Logotipo Oficial da Gráfica Máxima";
        logoImg.className = "img-responsive logo-empresa";
        container.appendChild(logoImg);
    } else {
        console.warn(`Container ${containerId} não
encontrado.`);
    }
}
```

ATENÇÃO Regra de Negócio: Ao integrar imagens provenientes de briefings de clientes (via IA para OCR), as mesmas não devem ser salvas no diretório estático /assets/img/. Elas devem ser processadas em memória (Blob/Base64) ou enviadas diretamente para um bucket temporário no backend (S3/Cloud Storage) para evitar sobrecarga no frontend.

5. Conclusão e Recomendações Finais

A estrutura de pastas fornecida é sólida e profissional. Para garantir a robustez do sistema:

Centralize as Chamadas: Todo acesso à API (seja backend próprio ou IA) deve passar por js/services/. Nenhum fetch direto dentro de modules/.

Padronize Erros: Use o errorHandler.js para converter erros técnicos (Ex: 500 Internal Server Error) em mensagens de negócio (Ex: "Não foi possível salvar o orçamento. Verifique sua conexão.").

Resiliência de IA: Implemente o padrão Circuit Breaker e Fallback para garantir que a gráfica nunca pare de trabalhar, mesmo se a OpenAI cair.

Monitoramento: Integre o errorHandler com uma ferramenta de monitoramento (Sentry, Datadog) para que você, como engenheiro, saiba dos erros antes mesmo do cliente reclamar.